

# Multimedia Application

By

Minhaz Uddin Ahmed, PhD

Department of Computer Engineering

Inha University Tashkent.

Email: [minhaz.ahmed@gmail.com](mailto:minhaz.ahmed@gmail.com)

# Content

- ▶ LLM
- ▶ Transformer
- ▶ Reason behind using Mixture Of Experts
- ▶ Training Dynamics & Scaling
- ▶ Inference, Prompting, & Techniques
- ▶ Challenges and Limitations
- ▶ Practice code

# LLM

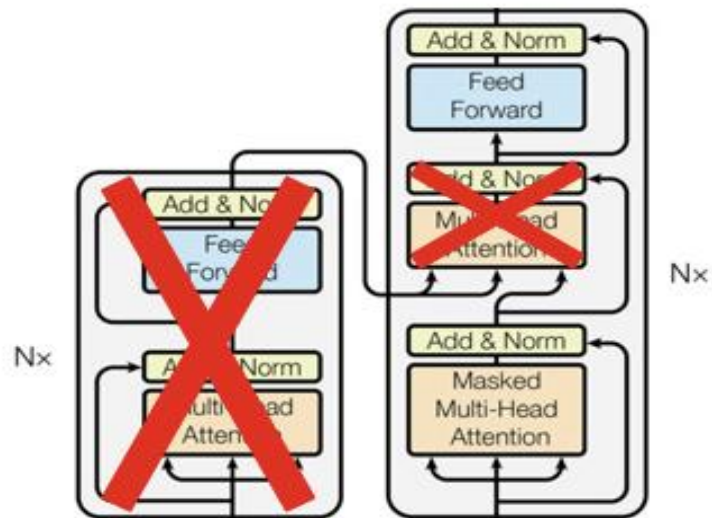
- ▶ LLM = Large Language Model
- ▶ "A language model is a statistical or machine learning model that **assigns probabilities** to **sequences of tokens**."

# Large

- ▶ Model size: billions of parameters or more
- ▶ Training data: 100s of billions of tokens or more
- ▶ Compute: a lot of GPUs

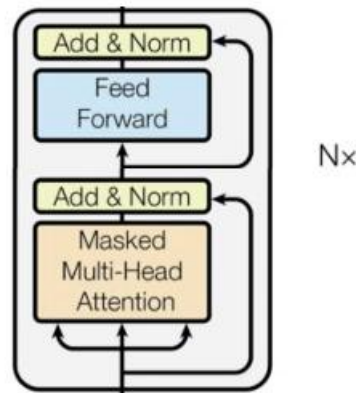
# Characteristics

Decoder-only Transformer-based model



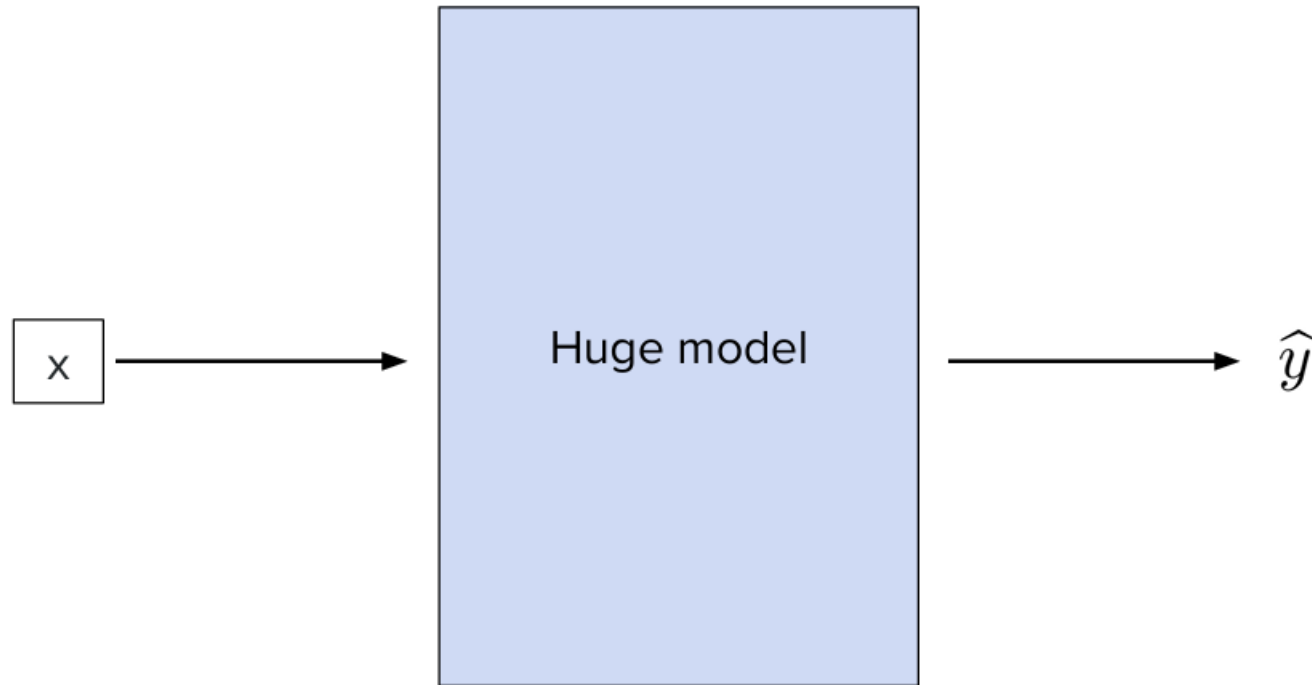
# Characteristics

- ▶ Decoder-only Transformer-based model



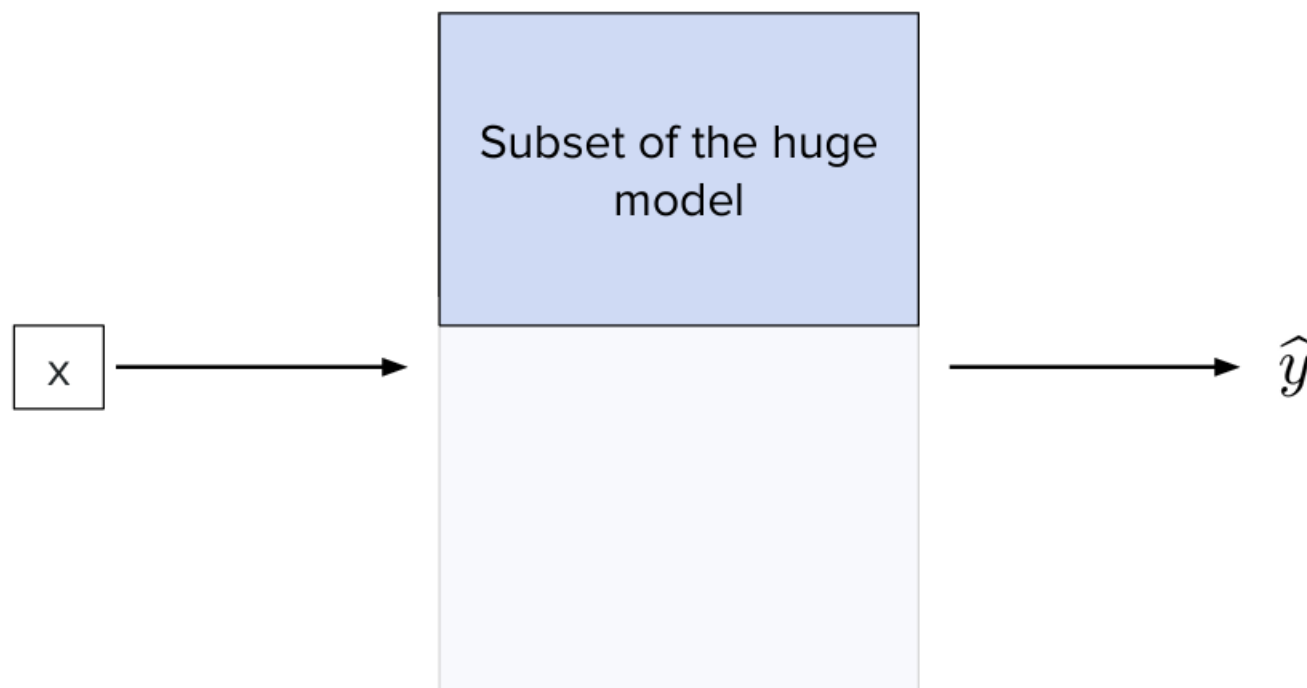
Examples: GPT series, LLaMA, Gemma, DeepSeek, Mistral, Qwen, ...

# Reason behind using Mixture Of Experts



Not all weights are useful in the forward pass

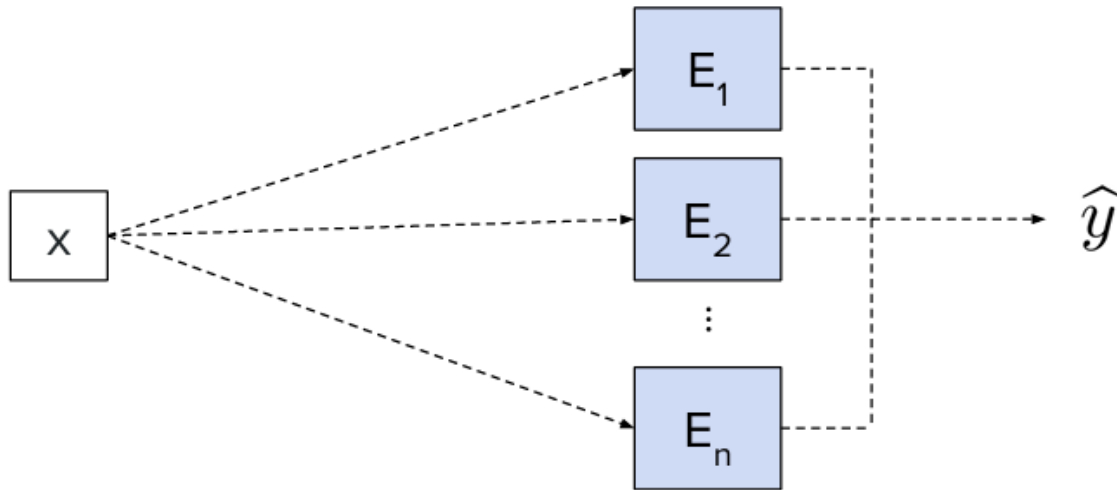
# Reason behind using Mixture Of Experts



Unlike dense models (e.g., Llama 3) that activate all parameters for every token, MoE models use a "sparse" approach—activating only a small, relevant subset of experts per input token. This allows models to scale to hundreds of billions or trillions of parameters while maintaining manageable computational costs during training and Inference.

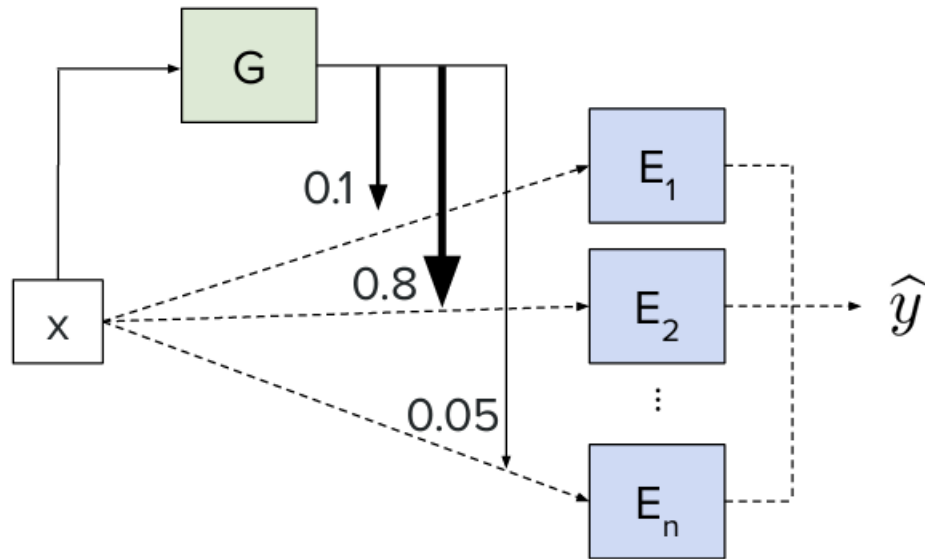


# Reason behind using Mixture Of Experts



A Mixture-of-Experts (MoE) based Large Language Model (LLM) is an architecture that improves model performance and efficiency by dividing tasks among multiple specialized sub-models, known as "experts," rather than using one massive, monolithic network.

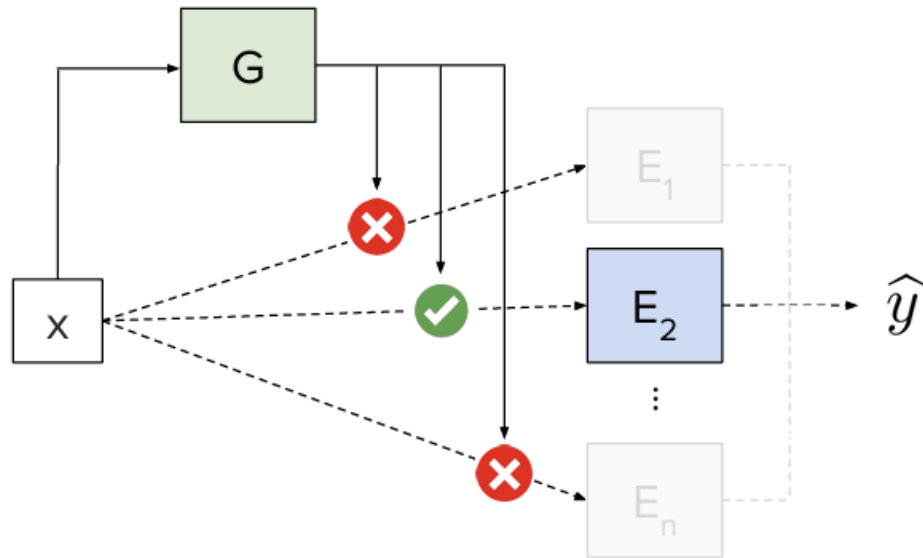
# Mixture Of Experts



**Dense MoE.** Output is weighted average of **all** expert outputs.

$$\hat{y} = \sum_{i=1}^n G(x)_i E_i(x)$$

# Mixture Of Experts

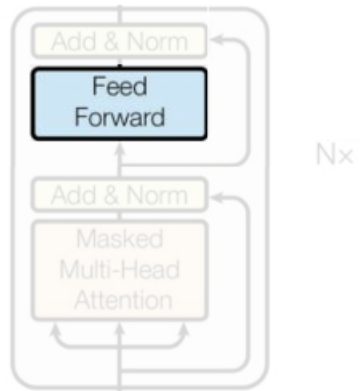


**Sparse MoE.** Output is weighted average of **selected** expert outputs.

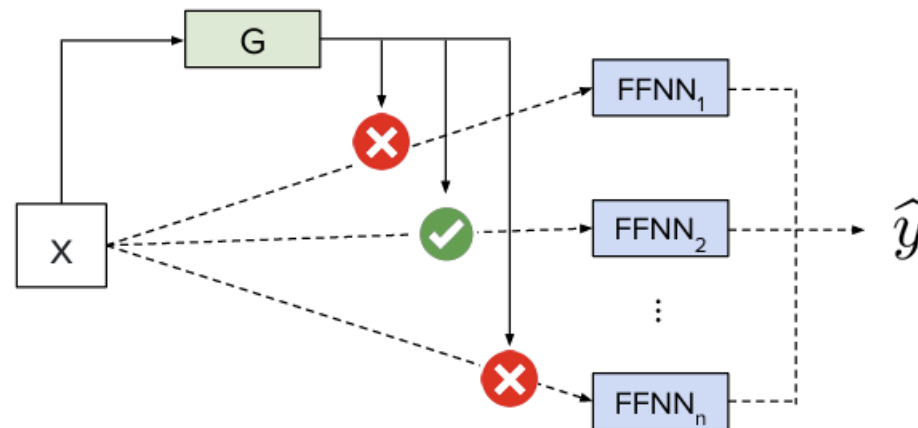
$$\hat{y} = \sum_{i \in \mathcal{I}_k} G(x)_i E_i(x)$$

Via **top-k** selection

# MOE in Transformer-based models



Routing done **for each token!**



Symptom. Same expert gets selected most of the time.

routing collapse

# Training challenges include routing collapse

**Remedy.** Force other experts to be "part of the game" too via auxiliary loss:

$$\text{loss} = \alpha \cdot N \cdot \sum_{i=1}^N f_i \cdot P_i$$

$f_i$  = fraction of tokens routed to expert i

$P_i$  = average routing probability for expert i

# Interpreting experts

## Layer 0

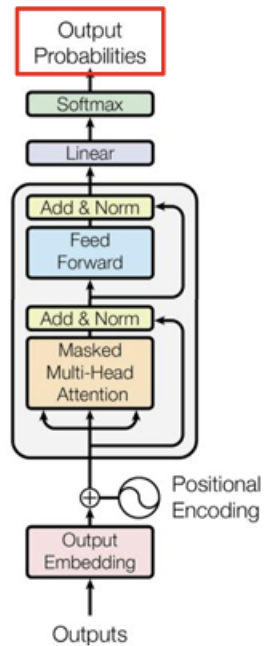
```
class MoeLayer(nn.Module):
    def __init__(self, experts: List[nn.Module],
                 gate: nn.Module, moe_args: Dict):
        super().__init__()
        assert len(experts) > 0
        self.experts = nn.ModuleList(experts)
        self.gate = gate
        self.args = moe_args

    def forward(self, inputs: torch.Tensor):
        inputs_squashed = inputs.view(-1, inputs.size(-1))
        gate_logits = self.gate(inputs_squashed)
        weights, selected_experts = torch.topk(
            gate_logits, self.args.num_experts_per_token
        )
        weights = nn.functional.softmax(
            weights,
            dim=-1,
            dtype=torch.float,
        ).type_as(inputs)
        results = torch.zeros_like(inputs_squashed)
        for i, expert in enumerate(self.experts):
            batch_idx, nth_expert = torch.where(
                selected_experts == i
            )
            results[batch_idx] += weights[batch_idx, nth_expert] * \
                expert(inputs_squashed[batch_idx])
        return results.view_as(inputs)
```

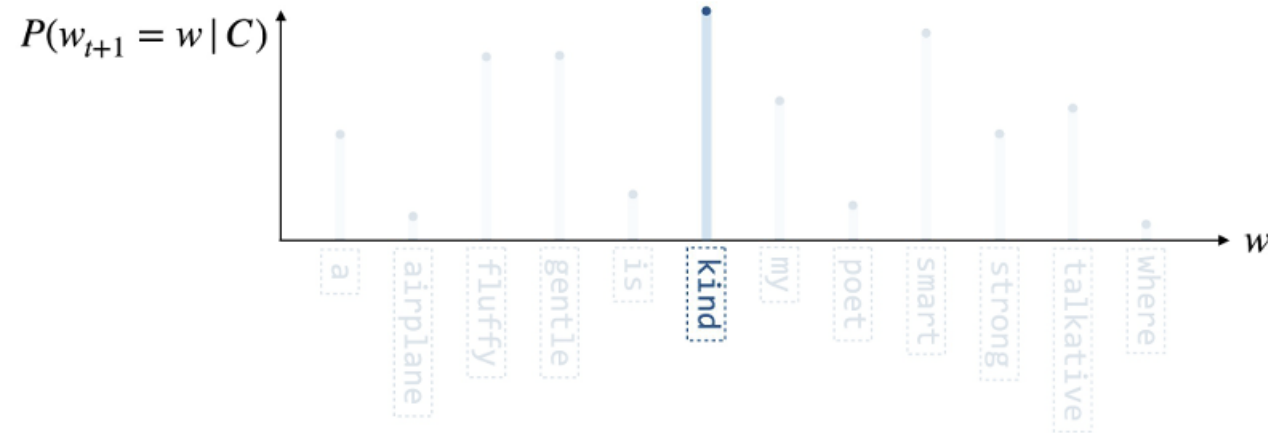
Each color represent an expert

# Response generation

- Predicting next token with **greedy decoding**



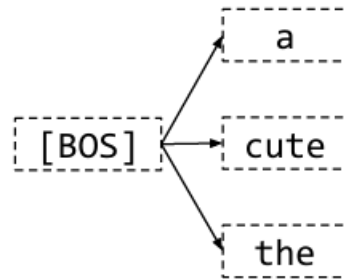
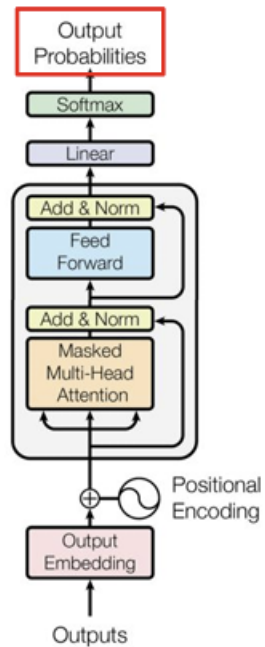
**1<sup>st</sup> idea.** Take token with highest predicted probability



**Limitations.** Output not optimal, natural and/or diverse

# Predicting next token with beam search

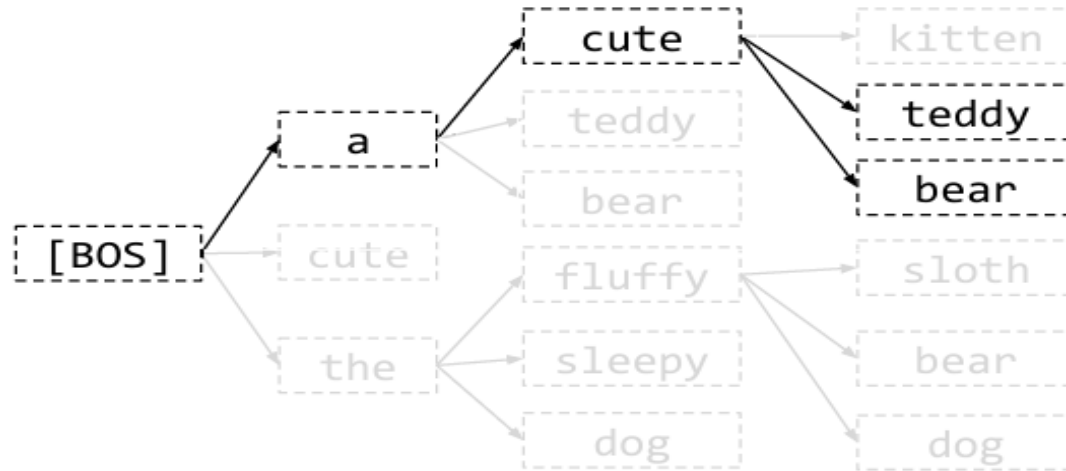
**2<sup>nd</sup> idea.** Keep k paths that are the most likely



Beginning of sequence



# Predicting next token with beam search



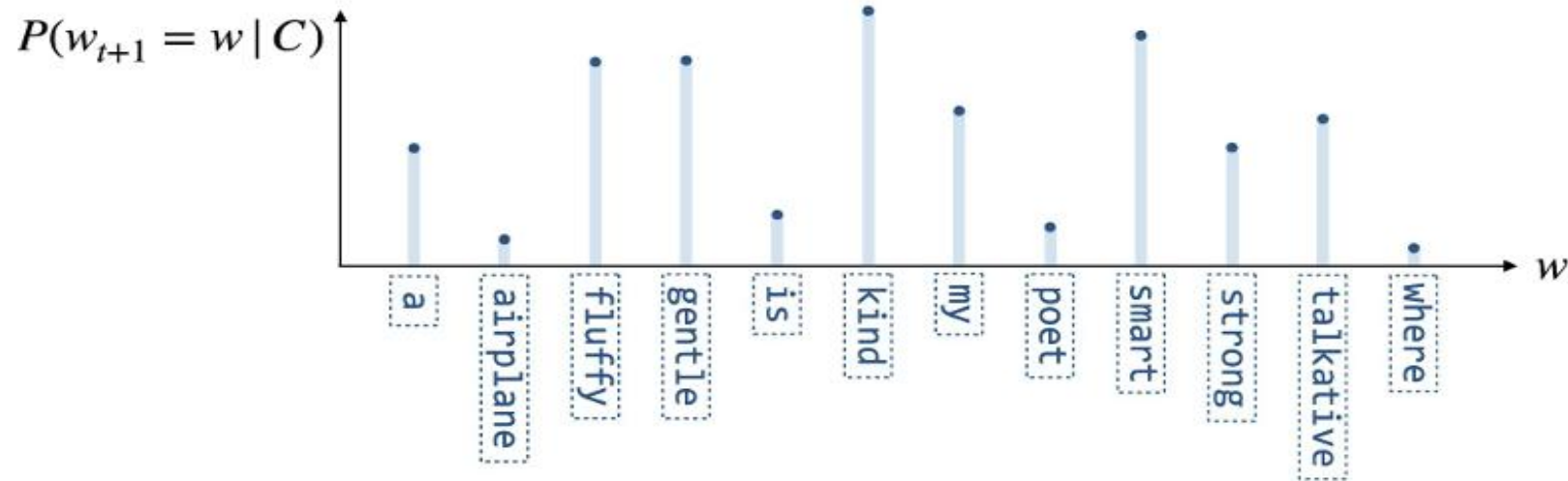
Ends at [EOS]

Keep k paths that are the most likely

# Predicting next token with sampling

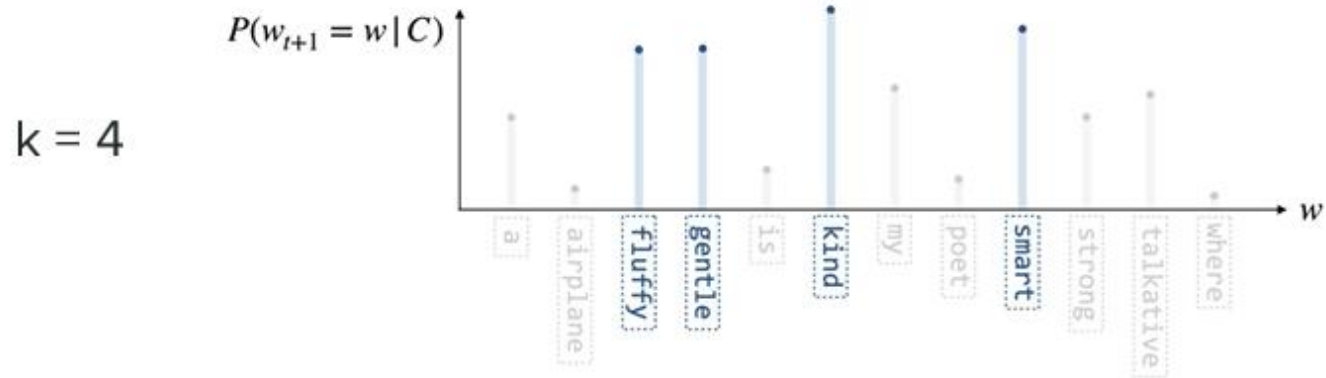
**3<sup>rd</sup> idea.** Sample next token from probability distribution:

$$\hat{w}_{t+1} \sim P(w_{t+1} | C)$$



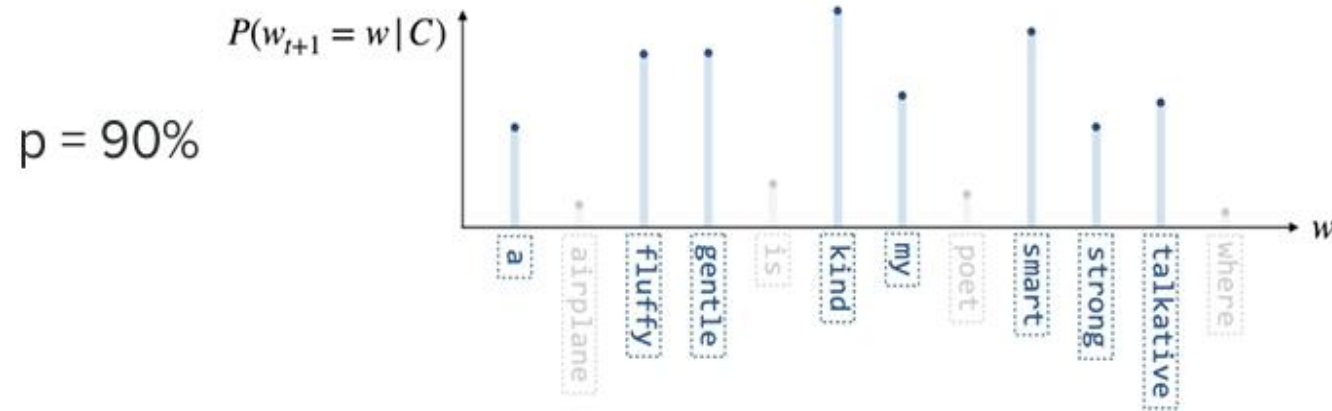
# Sampling strategies

- **Top-k:** Sample among top k most probable tokens



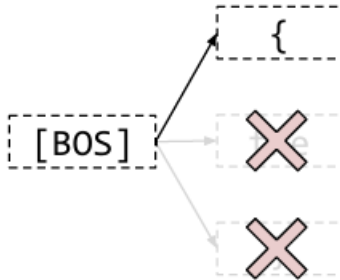
# Sampling strategies

- **Top-p:** Random sample among smallest set of tokens with cumulative probability  $\geq p$



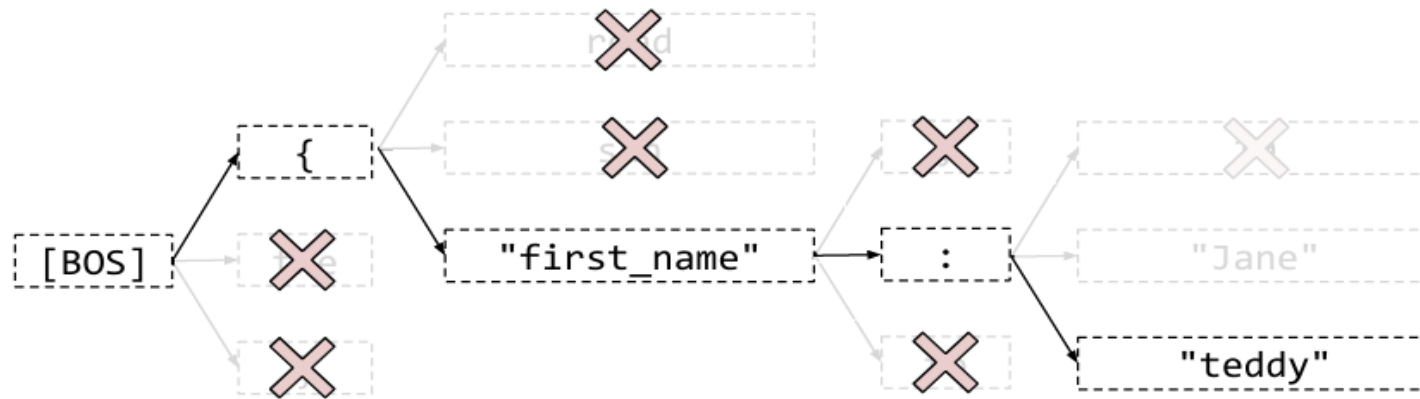
# Constraining the output via guided decoding

**Idea.** Only allow "valid" next tokens



# Constraining the output via guided decoding

**Idea.** Only allow "valid" next tokens



# Foundational lifecycle of modern LLM

- ▶ the foundational learning phase,
- ▶ the alignment process, and
- ▶ the empirical laws governing their performance.

# Pre-training: The "Base" Model

- ▶ Pre-training is the foundational phase where a model develops its understanding of syntax, semantics, factual knowledge, and reasoning capabilities. This is achieved through **Self-Supervised Learning (SSL)** on massive, unlabeled text corpora (e.g., Common Crawl, books, code repositories).
- ▶ The core objective is **Next Token Prediction**.
- ▶ The model uses a **Transformer architecture**, specifically utilizing multi-head self-attention mechanisms to weigh the importance of different parts of the input sequence relative to the token being predicted.



# Instruction Fine-Tuning & RLHF: Alignment

- ▶ Once the base model is trained, it must be aligned to act as an assistant rather than a completion engine. This process narrows the gap between the model's output and human intent.

**Supervised Fine-Tuning (SFT):** The model is trained on a curated dataset of "Instruction/Response" pairs (e.g., "Summarize this article: [Text]" -> "[Summary]"). This teaches the model the *format* of a helpful response.

# RLHF (Reinforcement Learning from Human Feedback):

- ▶ **Sampling:** The model generates multiple responses to a single prompt.
- ▶ **Ranking:** Human raters rank these responses from best to worst based on helpfulness, accuracy, and harmlessness.
- ▶ **Reward Model:** A secondary, smaller model is trained to predict what a human would prefer based on these rankings.
- ▶ **Optimization:** The primary model is then optimized using algorithms like PPO (Proximal Policy Optimization) to maximize the "Reward" assigned by the Reward Model.

# Scaling Laws: The Predictability of Power

- ▶ Scaling Laws, pioneered by researchers at OpenAI and DeepMind, suggest that performance is not random but follows power-law relationships between three primary inputs:

The Trinity of Scale:

- ▶ Parameters (N): The model's capacity to store information.
- ▶ Data (D): The variety and depth of tokens it is exposed to.
- ▶ Compute (C): The total floating-point operations (FLOPs) used during training.

**The Chinchilla Observation:** Researchers found that for a model to be optimal, the size of the dataset should grow proportionally with the model size. **If you scale parameters without scaling data, you hit diminishing returns quickly.**

# Prompting strategies

- ▶ Is Tashkent city beautiful ? → LLM → Yes

ICL = In-Context Learning

ICL a prompt engineering technique where LLMs learn new tasks from examples within the prompt, without updating their underlying weights or requiring retraining.

# In-context learning

| Zero-shot learning                                                                                                                                         | Few-shot learning                                                                                                                         |
|------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• Question is asked without examples</li><li>• Performance heavily depends on performance of initial model</li></ul> | <ul style="list-style-type: none"><li>• Prompt contains examples of input / output</li><li>• Typically has a better performance</li></ul> |

# In-context learning

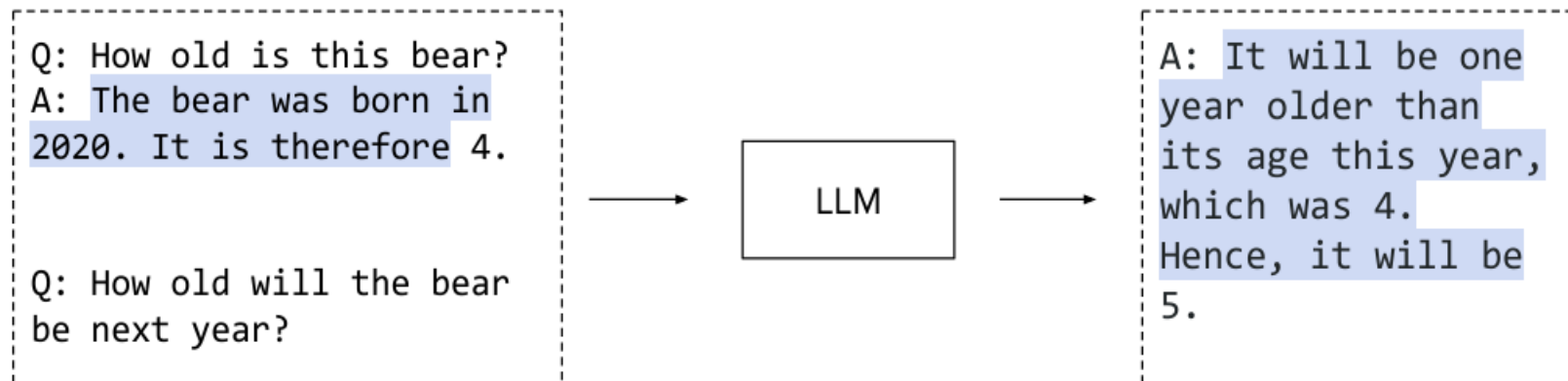
- ▶ Showing examples in the prompt is generally better but
- ▶ Requires effort
- ▶ Increases computational complexity & cost
- ▶ Increases latency

# Chain of thought

- ▶ A technique that improves **AI reasoning** by asking models to generate intermediate, step-by-step logical steps before providing a final answer.

# Chain of thought

**Idea.** Explaining reasoning helps in improving performance.



## Discussion.

- Interpretability + explanation
- More tokens: higher cost and latency

e.g. [Azizbek's graduation year from RAG](#)



# Self-consistency

Aggregating over reasoning paths improves performance.

- ▶ a prompt engineering method that enhances the reasoning capabilities of Large Language Models (LLMs) by generating multiple outputs and selecting the most consistent answer among them.
- ▶ E.g. gemini model provide multiple answers
- ▶ **Reduce hallucination**
- ▶ When a single misstep in a calculation ruins the whole result.

# Self-consistency

- ▶ The process follows three distinct stages:
- ▶ **Prompting with Chain-of-Thought (CoT):** The model is prompted to "think step-by-step."
- ▶ **Diverse Sampling:** By increasing the **temperature** setting, the LLM explores different logical paths. While the model might make an error in one path, it is statistically more likely to reach the correct answer through multiple different correct paths.
- ▶ **Majority Voting:** We aggregate all "Final Answers." The answer that appears most frequently is selected as the output, effectively filtering out one-off hallucinations.

# Self-consistency

- ▶ **Code Generation:** To find the most common structural approach to a problem.

# Inference optimizations

Computations are expensive, any way to reduce complexity?

- ▶ LLM inference optimizations, such as PagedAttention, quantization, and continuous batching, reduce latency and hardware costs while improving throughput. These techniques address bottlenecks in the prefill (prompt processing) and decode (token generation) phases by enhancing memory usage and computational efficiency

# Challenges

"Exact" efficiency:

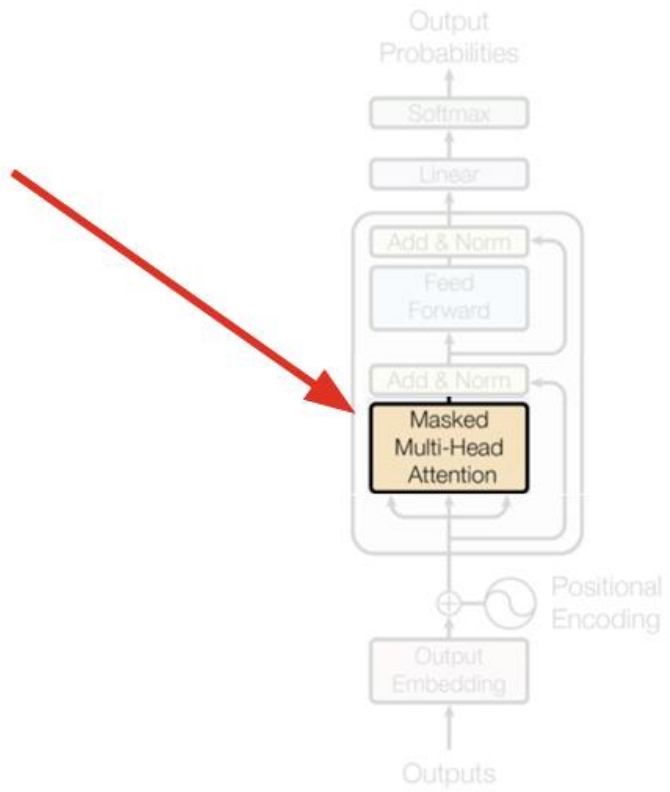
- ▶ Avoid redundancies
- ▶ Memory management
- ▶ Reformulate the math

# Challenges

Approximations:

- ▶ Architectural changes
- ▶ Embeddings representations
- ▶ Token prediction

# Attention-based



# Reusing computations with KV caching

- ▶ New token needs to interact with all previous tokens.

$$\text{softmax} \left( \frac{Q \boxed{K^T}}{\sqrt{d_k}} \right) \boxed{V}$$

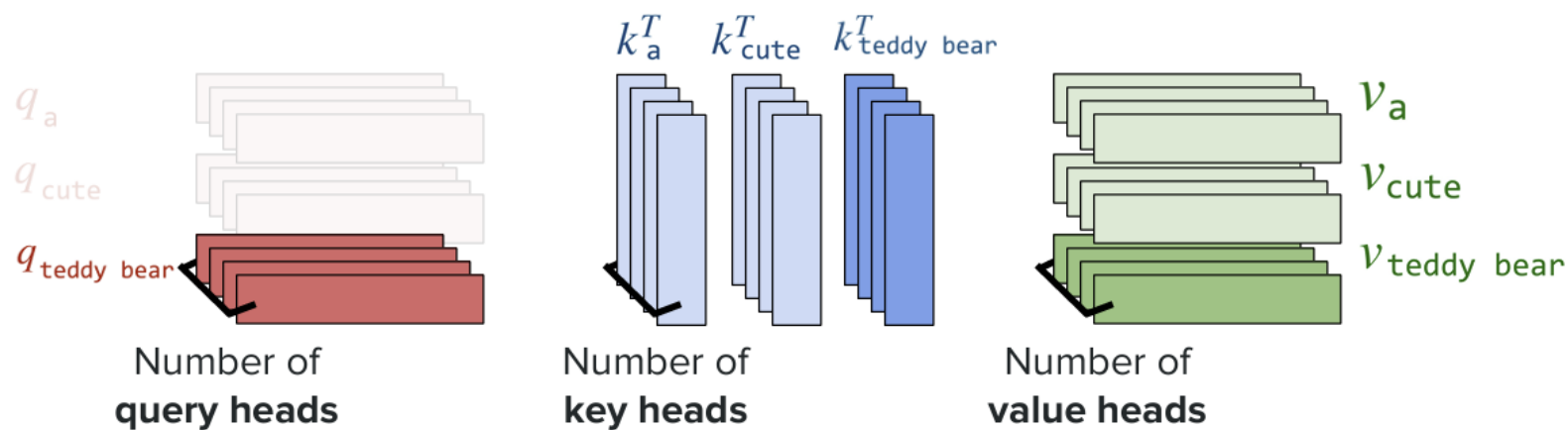
**KV Cache Management:** Reuses or offloads attention caches to handle long prompt contexts efficiently, as decoding is heavily memory-bound.

Keep keys and values in a cache.



# Reusing computations with KV caching

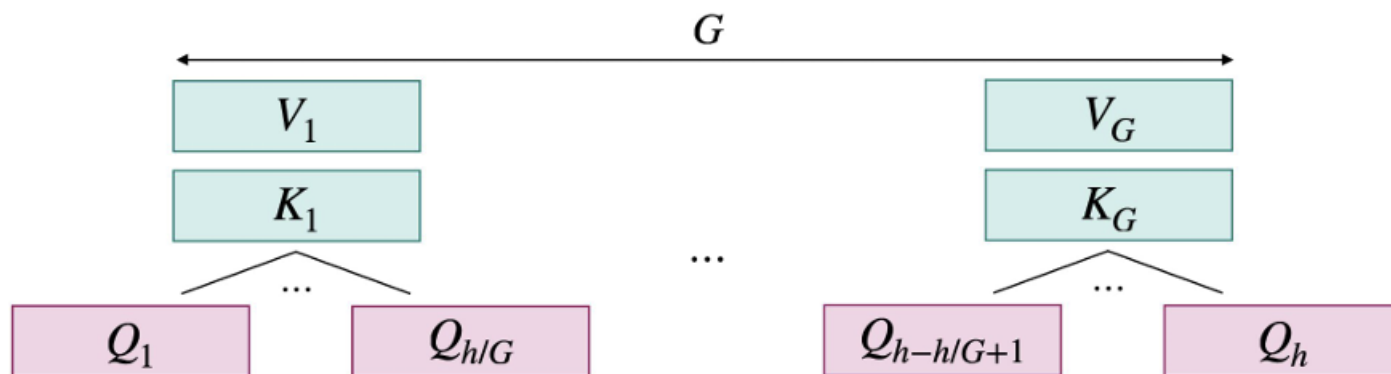
- **FlashAttention:** Optimizes the attention mechanism by reducing memory reads/writes between GPU high-bandwidth memory (HBM) and on-chip SRAM



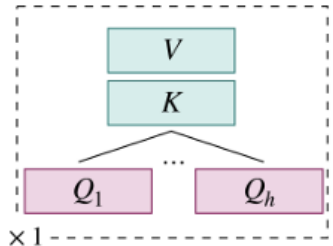
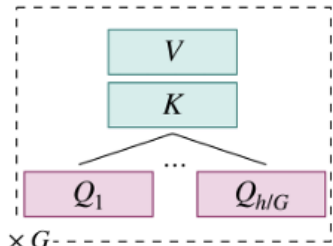
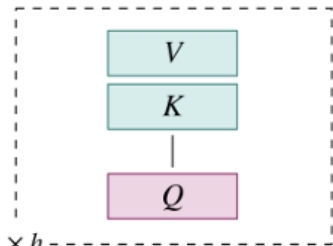
In vanilla MHA, Number of query heads = Number of key heads = Number of value heads =  $h$

# Sharing attention heads

Share key/value attention heads within groups of queries

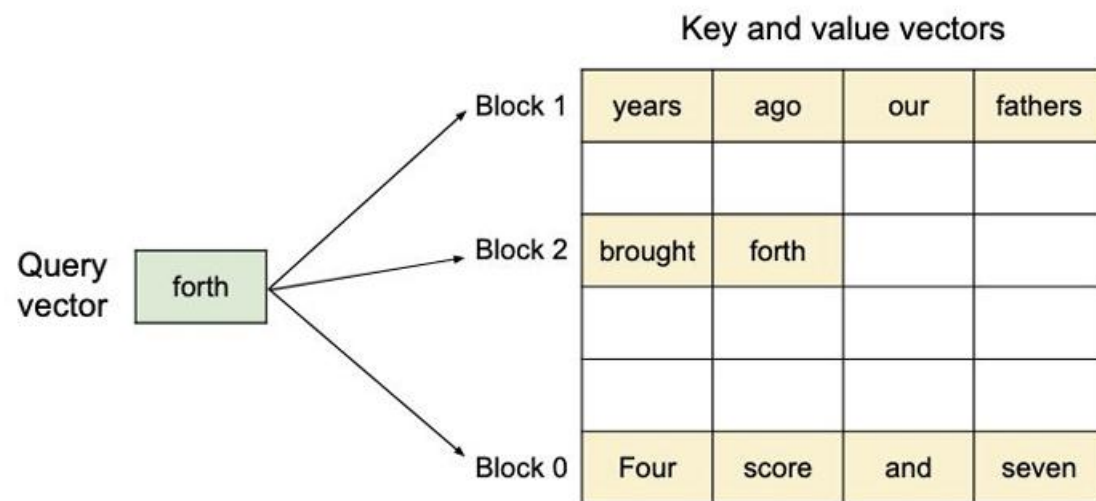


# Sharing attention heads

|             |                                       |                                                                                                    |
|-------------|---------------------------------------|----------------------------------------------------------------------------------------------------|
| $G = 1$     | <b>Multi-Query Attention</b><br>(MQA) | <br>$\times 1$  |
| $1 < G < h$ | <b>Group-Query Attention</b><br>(GQA) | <br>$\times G$  |
| $G = h$     | <b>Multi-Head Attention</b><br>(MHA)  | <br>$\times h$ |

# Manage memory with PagedAttention

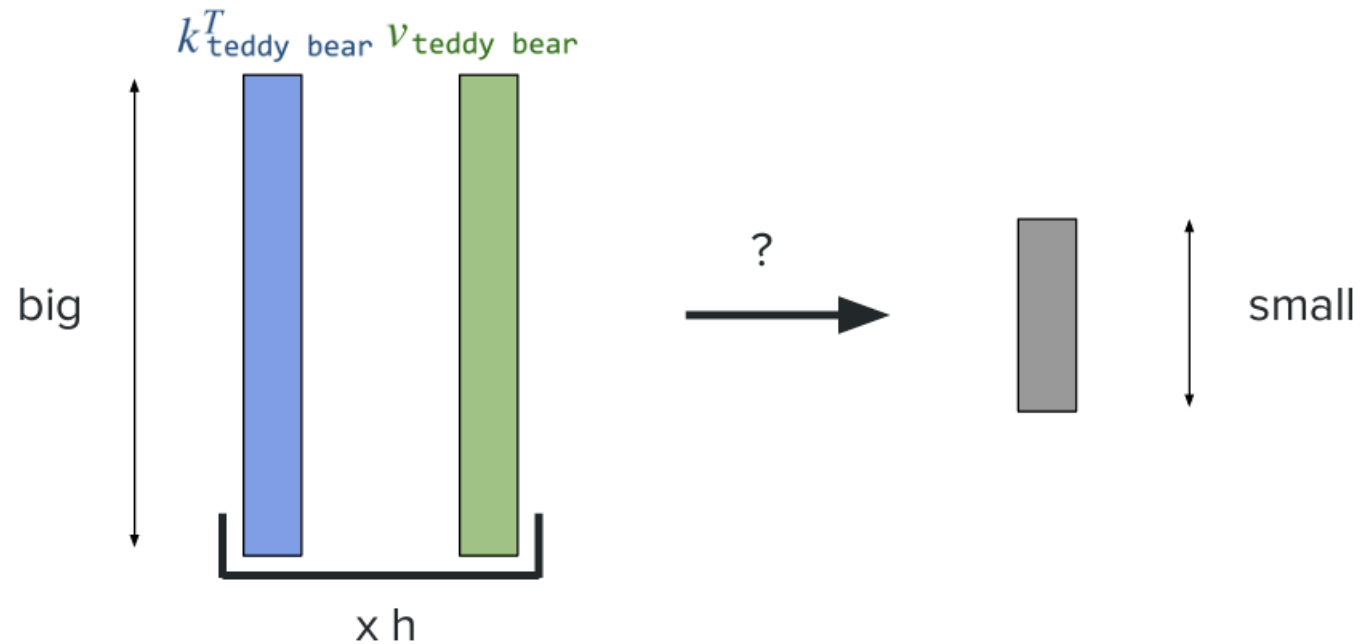
- ▶ Store K and V in non-contiguous space to minimize wasted memory



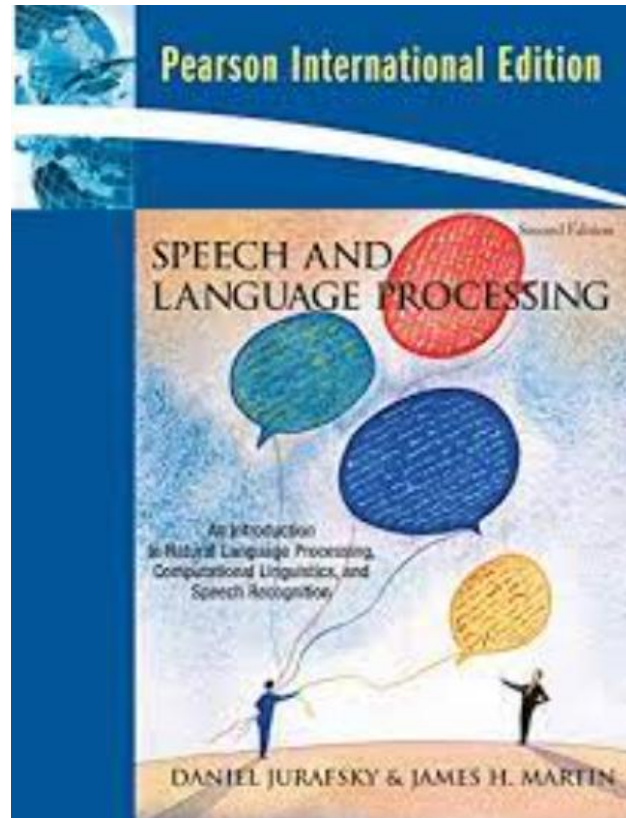
# Reduce memory of KV cache with latent attention

- ▶ Goal : Reduce dimension of K and V stored in memory

**Solution.** Store compressed representations instead!



# Reference



## Chapter 10

# Question



Thank you

